

Secure Software Development Processes

**Prepared by
Birce Duasu Hasçelik**

Department of Software Engineering

15 May 2026

TABLE OF CONTENTS

1. Introduction

2. Secure Software Development Life Cycle (SSDLC)

2.1 Requirements Phase

2.2 Design Phase

2.3 Implementation Phase

2.4 Testing Phase

2.5 Deployment Phase

2.6 Maintenance Phase

3. Threat Modeling

4. Secure Coding Practices

5. Tools and Techniques

6. Real-World Example: Secure Web Application (Login System)

7. Best Practices

Appendix A: Code Examples

8. Conclusion

References

1. Introduction

In today's digital world, software systems play a critical role in everyday life, from mobile applications to large-scale systems. As the use of software increases, security has become a major concern. Cyber attacks such as data breaches, unauthorized access, and injection attacks can lead to serious consequences including data loss and system damage.

Traditional software development mainly focuses on functionality and performance, often treating security as a secondary concern. However, this approach is no longer sufficient, as modern systems face constantly evolving threats. Unlike traditional development, secure software development integrates security into every stage of the process.

Secure software refers to software that is designed, developed, and maintained with security in mind. This means protecting data, preventing unauthorized access, and ensuring that the system behaves safely under different conditions.

Overall, considering security from the beginning helps reduce vulnerabilities and build more reliable and trustworthy systems.

2. Secure Software Development Life Cycle (SSDLC)

The Secure Software Development Life Cycle (SSDLC) is an approach that focuses on building security into every stage of software development. Instead of treating security as something to be checked at the end, SSDLC ensures that it is considered from the very beginning and throughout the entire process. This helps reduce vulnerabilities early and makes the system more reliable.

- **Requirements Phase:** It defines both functional and security needs of the system.
- **Design Phase:** It focuses on building a secure system architecture.
- **Implementation Phase:** It is where secure code is written.
- **Testing Phase:** It identifies security vulnerabilities before release.
- **Deployment Phase:** It ensures the system is securely released.
- **Maintenance Phase:** It focuses on keeping the system secure over time.

Overall, SSDLC emphasizes that security should not be an afterthought. By considering security at every stage, it becomes much easier to build software that is both functional and secure.

2.1 Requirements Phase

In this phase, the main goal is to determine what the system should do and what needs to be protected. Security requirements are identified alongside functional requirements. This includes defining user roles, identifying sensitive data and determining how authentication and data protection will be handled. Considering security at this early stage helps prevent major vulnerabilities later in the development process.

Example

In a web application with a login system, security requirements may include enforcing strong passwords, hashing user credentials before storing them and defining different access levels for users such as admin and regular users.

2.2 Design Phase

During this phase, the system structure is planned with security in mind. Developers analyze possible threats and design the system to reduce risks. One of the key activities in this stage is threat modeling where potential attack scenarios and weak points are identified. Based on this analysis, appropriate security measures are integrated into the system design.

Example

For a login system, a possible threat is brute-force attacks. To prevent this, the design may include limiting login attempts or temporarily locking accounts after multiple failed tries. APIs may also be designed to require authentication tokens to prevent unauthorized access.

2.3 Implementation Phase

In this phase, developers follow secure coding practices to avoid common vulnerabilities. This includes validating user input, properly handling sensitive data and avoiding unsafe coding techniques. Many security issues originate from coding mistakes so writing secure code is critical for building a reliable system.

Example

Instead of directly inserting user input into SQL queries, developers use prepared statements to prevent SQL injection attacks. Additionally, user inputs are validated to ensure only expected data is processed.

2.4 Testing Phase

In this stage, the system is tested not only for functionality but also for security. Different testing methods are used to detect vulnerabilities in both the source code and the running application. Identifying and fixing issues at this stage is more efficient and less costly than addressing them after deployment.

Example

Static testing tools can analyze the source code to detect insecure patterns while dynamic testing can simulate attacks such as sending malicious inputs to forms and observing how the system responds.

2.5 Deployment Phase

This phase involves preparing the application for production and ensuring that it is securely configured. Even if the software is developed securely, improper configuration can introduce vulnerabilities. Therefore, secure settings and environment configurations must be carefully applied.

Example

A web application should use HTTPS to encrypt communication. In addition, firewalls should be configured, unnecessary services should be disabled and access to sensitive resources should be restricted.

2.6 Maintenance Phase

Security is an ongoing process so the system must be continuously monitored and updated after deployment. This includes applying security patches, updating dependencies and reviewing logs to detect unusual activity. As new threats emerge, the system should be adapted to maintain its security.

Example

If a vulnerability is discovered in a library used by the application, it should be updated immediately. Monitoring system logs can also help detect suspicious behavior such as repeated failed login attempts

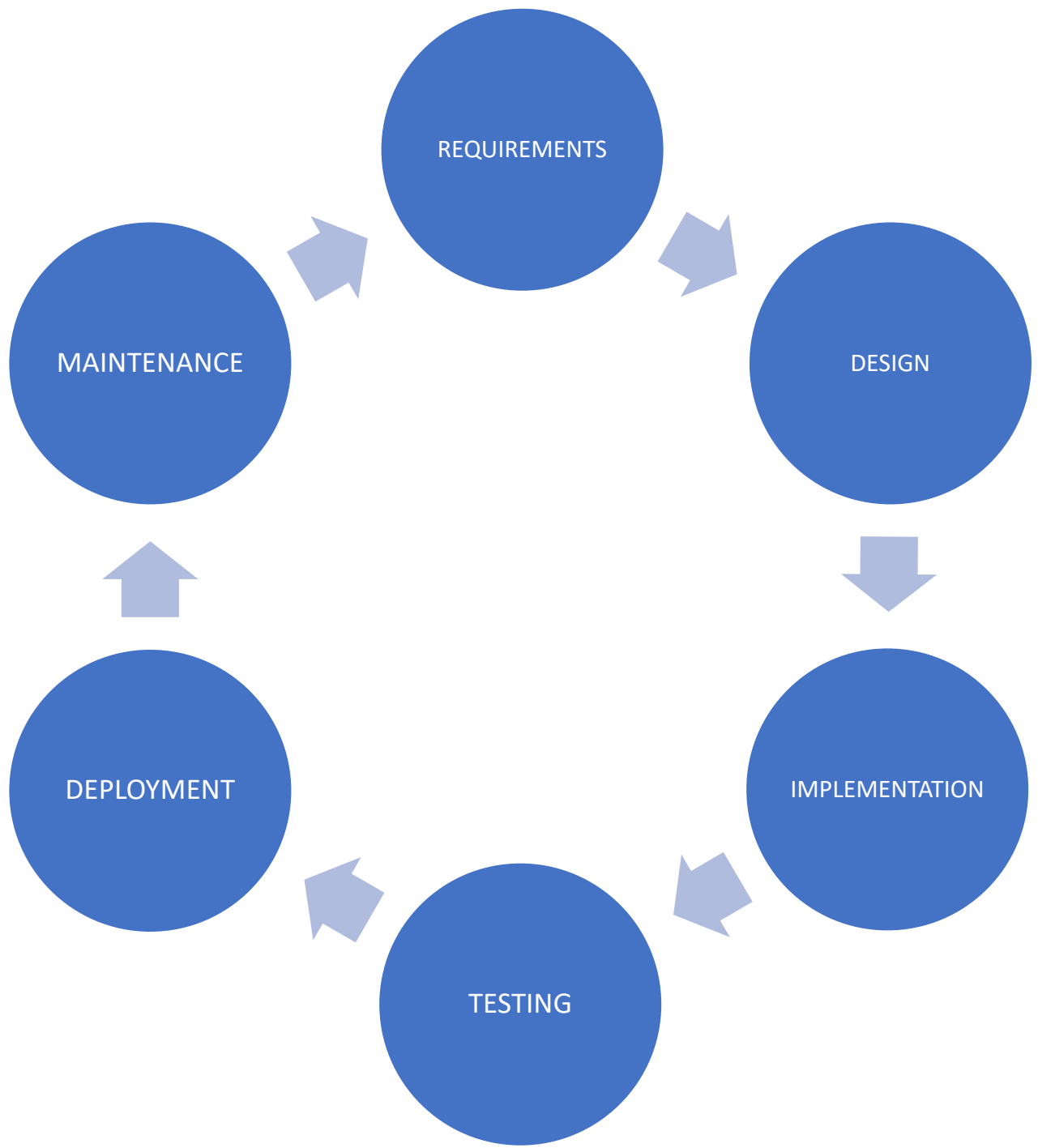


Figure 1. Secure Software Development Life Cycle (SSDLC)

3. Threat Modeling

Threat modeling is a structured approach used during the design phase of software development to anticipate possible attacks and identify weak points in the system. Instead of reacting to security issues after they occur, this approach allows developers to think like attackers and take preventive measures early.

One widely used method in threat modeling is the STRIDE model which categorizes threats into six types:

- **Spoofing**: pretending to be another user or system
- **Tampering**: modifying data without authorization
- **Repudiation**: denying an action or transaction
- **Information Disclosure**: exposing sensitive data
- **Denial of Service (DoS)**: making the system unavailable
- **Elevation of Privilege**: gaining higher access rights

By analyzing these categories, developers can identify risks and implement appropriate security controls. This process also helps prioritize threats based on their impact and likelihood.

Example

Consider a web application with a login system:

- **Spoofing**: An attacker tries to log in as another user using stolen credentials
- **Tampering**: Data in a request is modified before reaching the server
- **Repudiation**: A user denies performing an action such as claiming they did not make a transaction, due to lack of proper logging
- **Information Disclosure**: User passwords are exposed due to insecure storage
- **Denial of Service**: Multiple login attempts are used to overload the system
- **Elevation of Privilege**: A normal user tries to access admin functionalities

To prevent these threats, measures such as strong authentication, input validation, encryption, and access control should be implemented.

4. Secure Coding Practices

Secure coding focuses on reducing risks caused by improper handling of data and system behavior. Developers must ensure that all inputs are properly validated, sensitive operations are protected, and the system behaves safely even in unexpected situations.

One of the most important aspects is **input validation**, where all user inputs are checked before being processed. This helps prevent attacks such as SQL injection and cross-site scripting (see Appendix A).

Another key area is **authentication and authorization**, which ensures that users are properly identified and only allowed to access resources they are permitted to use.

Error handling is also critical. Systems should not reveal sensitive information through error messages, as this can help attackers understand the system structure.

Finally, **logging** helps monitor system activity and detect suspicious behavior. Proper logs can be used to analyze attacks and improve system security.

Example

- **Input Validation:** A login form should validate user input to ensure it does not contain malicious data. For example, restricting input length and rejecting unexpected characters helps prevent injection attacks.
- **Authentication and Authorization:** After login, users should only access features based on their roles. For instance, an admin panel should not be accessible to regular users.
- **Error Handling:** Instead of showing detailed database errors to users, the system should display a generic message like “Something went wrong” while logging the actual error internally.
- **Logging:** Failed login attempts should be recorded in the system logs. Repeated failures from the same user or IP address may indicate a brute-force attack.

Secure coding practices play a critical role in preventing vulnerabilities at the source of software development.

5. Tools and Techniques

Various tools and techniques are used to ensure security at different stages of development. One of the most common approaches is using automated testing tools to identify vulnerabilities early.

Static Application Security Testing (SAST) tools analyze the source code without executing the program. These tools help detect coding issues such as insecure functions, hardcoded credentials, and injection vulnerabilities during the development phase.

On the other hand, **Dynamic Application Security Testing (DAST)** tools test the application while it is running. They simulate real-world attacks and identify vulnerabilities such as broken authentication or misconfigurations.

Another important concept is integrating security into the development pipeline through **CI/CD (Continuous Integration / Continuous Deployment)**. By adding security checks into the pipeline, vulnerabilities can be detected automatically whenever new code is added, ensuring continuous security throughout the project.

Example

- **SAST Example:**
A tool like SonarQube scans the source code and detects insecure coding patterns, such as missing input validation or potential SQL injection risks.
- **DAST Example:**
A running web application is tested by sending malicious inputs to forms to see if the system properly handles them.
- **CI/CD Example:**
When a developer pushes new code, automated security checks run in the pipeline. If a vulnerability is detected, the build may fail until the issue is fixed.

Security tools and automated techniques help identify vulnerabilities early and ensure continuous protection throughout the development lifecycle.

Feature	SAST	DAST
Full Name	Static Application Security Testing	Dynamic Application Security Testing
Testing Method	Analyzes source code without execution	Tests the application during execution
Development Stage	Early development phase	Testing or deployment phase
Main Purpose	Detects coding vulnerabilities	Detects runtime vulnerabilities
Source Code Requirement	Requires source code access	Does not require source code
Example Tools	SonarQube, Checkmarx	OWASP ZAP, Burp Suite
Main Advantage	Detects issues early in development	Simulates real-world attacks
Main Limitation	Cannot detect runtime issues	Cannot fully analyze internal source code
Common Vulnerabilities Detected	SQL injection risks, insecure functions, hardcoded credentials	Authentication issues, configuration problems, runtime security flaws

6. Real-World Example: Secure Web Application (Login System)

In a secure web application, the login system must be designed to protect user credentials and prevent unauthorized access. This requires applying multiple security practices across different stages of development.

First, user credentials should be handled securely. Passwords must never be stored in plain text; instead, they should be hashed using secure algorithms such as bcrypt (see Appendix A). This ensures that even if the database is compromised, attackers cannot easily retrieve original passwords.

Input validation is also essential in a login system. All user inputs must be checked to ensure they do not contain malicious data. Without proper validation, the system may become vulnerable to attacks such as SQL injection.

Authentication and authorization must be properly implemented. After verifying user credentials, the system should generate a secure session or token to manage user access. Additionally, different user roles should be enforced to control access to sensitive features.

Secure communication is another key aspect. Data transmitted between the user and the server must be encrypted using HTTPS to prevent interception.

Finally, the system should include protection mechanisms against common attacks. These include limiting login attempts, detecting suspicious behavior, and logging failed access attempts.

Example

- **Password Security:**
User passwords are hashed before being stored in the database using algorithms like bcrypt.
- **Input Validation:**
The login form checks user inputs to ensure they do not contain unexpected or harmful data.
- **SQL Injection Prevention:**
Prepared statements are used instead of directly inserting user input into SQL queries.
- **Authentication & Session Management:**
After login, the system creates a secure session ID or token to identify the user.
- **HTTPS Usage:**
All communication between client and server is encrypted to protect sensitive data.
- **Brute-force Protection:**
The system limits login attempts or temporarily locks the account after multiple failed tries.
- **Logging:**
Failed login attempts are recorded to detect suspicious activity.

7. Best Practices

Following best practices ensures that security is consistently applied across all stages of software development. These principles are not tied to a single phase but guide the entire process.

One important principle is **security by design**, which means considering security from the beginning rather than adding it later. This approach helps reduce vulnerabilities and ensures that security is part of the system architecture.

Another key principle is **least privilege**, where users and systems are given only the minimum level of access necessary. This reduces the potential damage if an account is compromised.

Defense in depth is also widely used, meaning multiple layers of security are applied instead of relying on a single protection mechanism. Even if one layer fails, others continue to protect the system.

Keeping systems updated is another essential practice. **Regular updates and patching** help fix known vulnerabilities and protect the system against newly discovered threats.

Finally, **code review** plays an important role in improving software security. Reviewing code helps detect vulnerabilities, improve code quality, and ensure that secure coding standards are followed.

Example

- **Security by Design:**
A system is designed with authentication and encryption from the beginning instead of adding them after development.
- **Least Privilege:**
A normal user is not allowed to access admin panels or sensitive system settings.
- **Defense in Depth:**
A web application uses input validation, authentication, and firewall protection together instead of relying on only one security measure.
- **Regular Updates:**
A system regularly updates its libraries to fix known security vulnerabilities.
- **Code Review:**
Developers review each other's code to detect potential security issues before deployment.

Applying best practices helps ensure that security is not dependent on a single mechanism but is consistently enforced throughout the system.

Appendix A: Code Examples

SQL Injection Prevention

```
PreparedStatement stmt = connection.prepareStatement(  
    "SELECT * FROM users WHERE username = ?");  
stmt.setString(1, username);
```

Password Hashing

```
String hashedPassword = BCrypt.hashpw(password, BCrypt.gensalt());
```

Password Verification

```
if(BCrypt.checkpw(inputPassword, storedHash)){  
    System.out.println("Login successful");  
}
```

Input Validation

```
if(username.matches("[a-zA-Z0-9]+")){  
    // valid input  
}
```

Basic Error Handling

```
try {  
    // login logic  
} catch(Exception e){  
    System.out.println("Something went wrong");  
}
```

8. Conclusion

In conclusion, secure software development is an essential part of building reliable and trustworthy systems in today's digital world. As cyber threats continue to evolve, it is no longer sufficient to consider security only at the final stages of development. Instead, security must be integrated into every phase of the software development life cycle.

Throughout this report, the Secure Software Development Life Cycle (SSDLC) has been examined, highlighting how security can be applied from requirements to maintenance. Each phase plays an important role in identifying risks, reducing vulnerabilities, and ensuring that the system remains protected over time.

In addition, concepts such as threat modeling, secure coding practices, and the use of security tools demonstrate how potential attacks can be anticipated and prevented. Real-world examples, such as a secure login system, further show how these practices can be applied in practical scenarios.

Finally, following best practices such as security by design, least privilege, and regular updates helps create a strong security foundation. Overall, integrating security into the development process not only protects systems and data but also increases user trust and system reliability.

References

OWASP Foundation. (2023). *OWASP Top Ten Web Application Security Risks*. Retrieved from <https://owasp.org>

Microsoft. (2022). *Microsoft Security Development Lifecycle (SDL)*. Retrieved from <https://learn.microsoft.com>

NIST. (2020). *Secure Software Development Framework (SSDF)*. National Institute of Standards and Technology.

McGraw, G. (2006). *Software Security: Building Security In*. Addison-Wesley.

Howard, M., & Lipner, S. (2006). *The Security Development Lifecycle*. Microsoft Press.

Stuttard, D., & Pinto, M. (2011). *The Web Application Hacker's Handbook*. Wiley.